Image Segmentation Using Graph Cut Technique

Gattu Charitha B23EE1021

April 9, 2026

Abstract

Image segmentation partitions an image into meaningful regions. In this work, we implement a Graph Cut based method that formulates segmentation as an energy minimisation problem.User-provided scribbles define foreground and background regions. Based on these, pixel-wise data costs are computed using colour similarity, while smoothness constraints enforce spatial consistency between neighbouring pixels.

The image is represented as a graph where pixels are nodes and edges encode data and smoothness terms. The final segmentation is obtained using the min-cut/max-flow algorithm.The method produces smooth and accurate results, with post-processing used to reduce noise and refine boundaries.

Contents

1 Introduction **2**

2 Data Preparation and Annotation **2**
2.1 Input Images . . . 2
2.2 User Scribbles . . . 3

3 Graph Construction **3**
3.1 Terminal Edges (t-links) . . . 3
3.2 Neighbour Edges (n-links) . . . 3
3.3 Min-Cut / Max-Flow Optimization . . . 4

4 Energy Formulation **5**
4.1 Data Term (E_{data}) . . . 5
4.2 Smoothness Term (E_{smooth}) . . . 5
4.3 Foreground–Background Modelling . . . 6

5 Segmentation Algorithm **6**
5.1 Iterative Refinement . . . 7

6 Results **7**

7 Post-Processing **8**

8 Discussion **10**

9 Conclusion **10**

1 Introduction

Image segmentation refers to the process of dividing an image into distinct regions that correspond to different objects or parts of objects. It serves as a fundamental step in many computer vision applications such as object recognition, medical image analysis, and scene understanding.

Traditional methods like thresholding or clustering operate purely on pixel intensity or colour values, ignoring spatial relationships. As a result, they often produce noisy and fragmented outputs.

Graph Cut methods overcome this limitation by modelling the image as a graph and incorporating both:

- Pixel-level information (data term)
- Neighbourhood relationships (smoothness term)

This allows the algorithm to produce globally optimal segmentations that are both accurate and smooth.

2 Data Preparation and Annotation

2.1 Input Images

The segmentation pipeline is applied to a set of images with varying characteristics such as:

- Differences in colour distribution
- Varying object shapes and boundaries
- Different background complexities



(a) Image 1 (b) Image 2 (c) Image 3

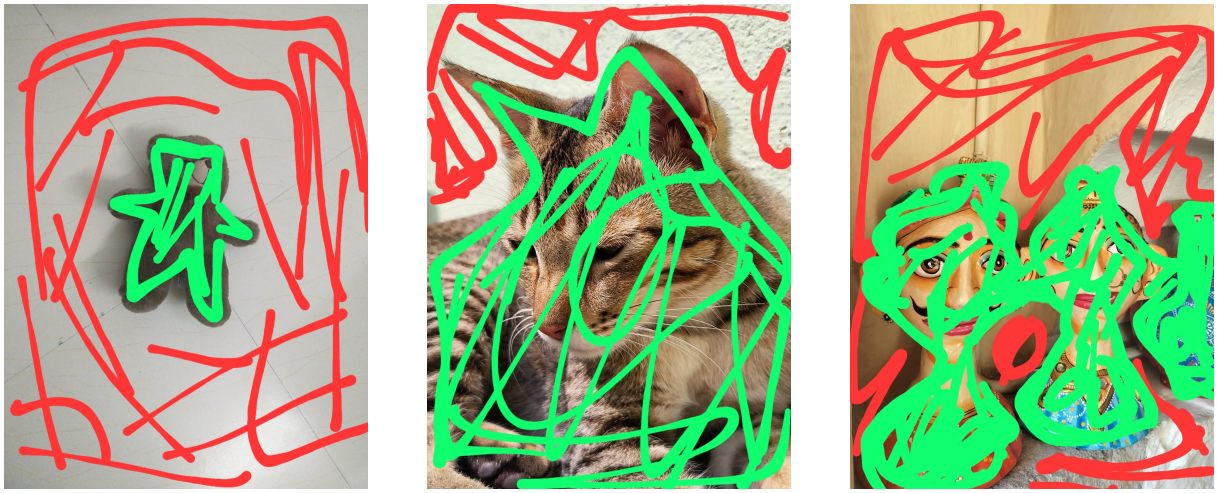
Figure 1: Side-by-side images

2.2 User Scribbles

User annotations are provided in the form of scribbles:

- Foreground scribbles mark the object of interest
- Background scribbles mark non-object regions

These annotations serve as strong constraints and guide the segmentation process.



(a) Scribble 1 (b) Scribble 2 (c) Scribble 3

Figure 2: User-provided scribble annotations for segmentation

3 Graph Construction

The image is represented as a graph $G = (V, E)$ where:

- Each pixel corresponds to a node in V
- Two special nodes are added: source (foreground) and sink (background)

Edges are defined in two categories:

3.1 Terminal Edges (t-links)

Each pixel is connected to the source and sink nodes. These edges represent the cost of assigning a pixel to foreground or background.

Pixels similar to foreground scribbles have lower cost to connect to the source, while pixels similar to background have lower cost to connect to the sink.

3.2 Neighbour Edges (n-links)

Neighbour edges connect adjacent pixels and enforce spatial smoothness. The strength of these edges depends on the similarity between neighbouring pixels.

If two neighbouring pixels have similar colour values, the connection between them is strong, discouraging label changes. If they are very different, the connection is weak, allowing boundaries to form.

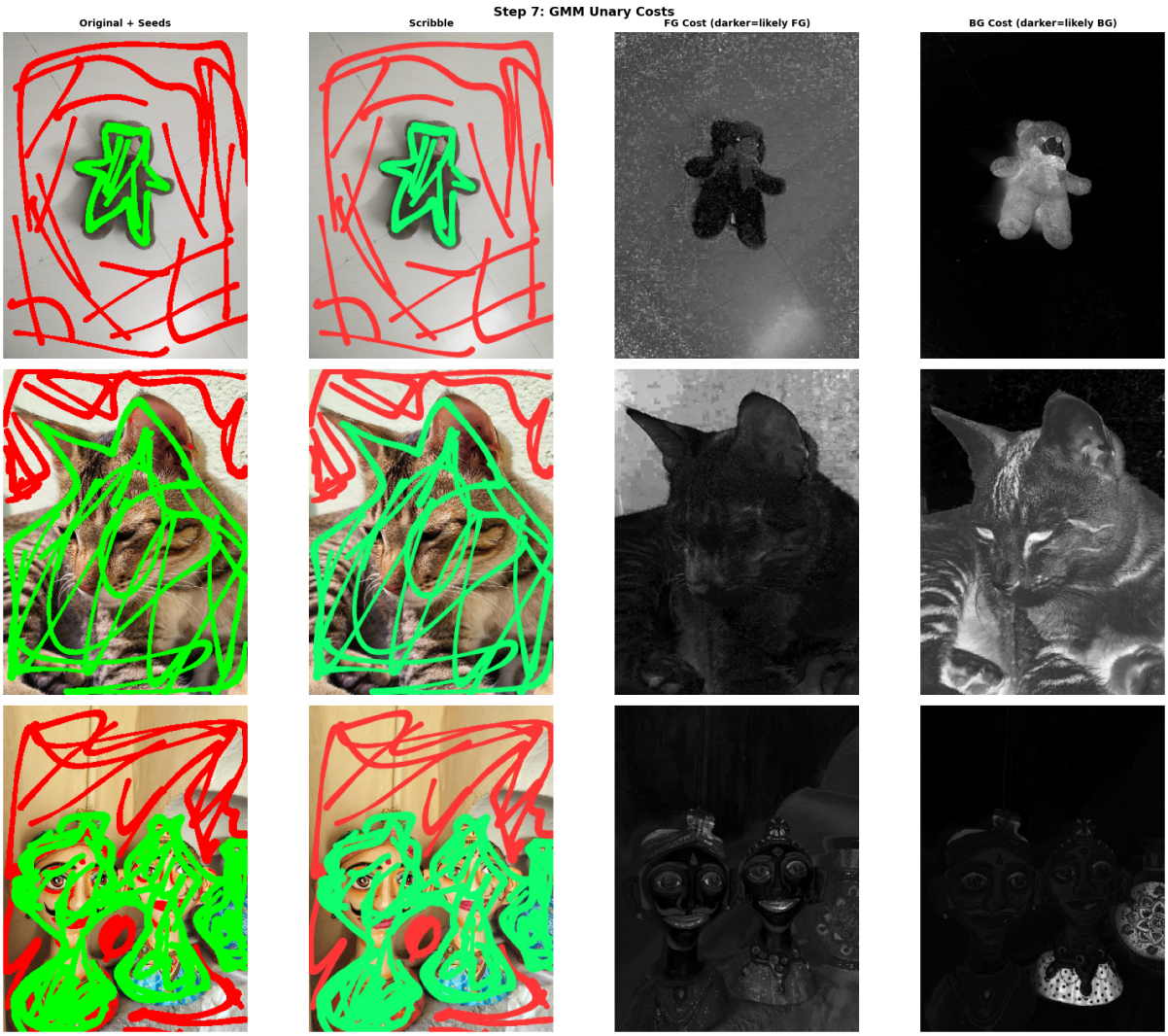


Figure 3: Visualization of the segmentation pipeline. The first column shows the original images with user-provided foreground (green) and background (red) scribbles. The second column shows the refined scribble masks. The third column represents the foreground (FG) unary cost maps, where darker regions indicate higher likelihood of belonging to the foreground. The fourth column shows background (BG) unary cost maps, where darker regions indicate higher likelihood of belonging to the background.

3.3 Min-Cut / Max-Flow Optimization

Once the graph is constructed, the segmentation problem is solved using the min-cut / max-flow algorithm. The algorithm finds the minimum cost cut that separates the source (foreground) node from the sink (background) node.

The cut determines the optimal labeling of pixels:

- Pixels connected to the source belong to the foreground.
- Pixels connected to the sink belong to the background.

This cut corresponds to the segmentation that minimises the energy function defined earlier.

4 Energy Formulation

The segmentation problem is formulated as minimising an energy function that balances data fidelity with spatial smoothness. The goal is to assign each pixel either a foreground or background label in such a way that the overall segmentation is consistent with the image appearance while avoiding noisy or fragmented regions.

The energy function is defined as:

$$E(L) = \sum_{p \in V} D_p(L_p) + \lambda \sum_{(p,q) \in N} w_{pq} \cdot [L_p \neq L_q]$$

where:

- L_p represents the label assigned to pixel p (foreground or background)
- $D_p(L_p)$ is the data cost of assigning label L_p
- w_{pq} represents the similarity weight between neighbouring pixels p and q
- λ controls the strength of the smoothness constraint
- $[L_p \neq L_q]$ is an indicator function that equals 1 when the labels differ

where E_{data} represents the data term and E_{smooth} represents the smoothness term. Minimising this energy produces the most optimal segmentation according to these two criteria.

4.1 Data Term (E_{data})

The data term measures how well a pixel fits into the foreground or background class based on the user-provided annotations. Pixels whose colours are similar to the foreground scribbles are assigned a lower cost for being labelled as foreground, while pixels similar to the background scribbles are assigned a lower cost for being labelled as background.

This term ensures that the segmentation result remains consistent with the appearance information provided by the user annotations.

4.2 Smoothness Term (E_{smooth})

The pairwise smoothness weight between neighbouring pixels is defined as:

$$w_{pq} = \exp\left(-\frac{\|I_p - I_q\|^2}{2\sigma^2}\right)$$

where I_p and I_q represent the colour vectors of neighbouring pixels. This formulation encourages neighbouring pixels with similar colour values to share the same label while allowing boundaries to form where strong intensity differences exist.

4.3 Foreground–Background Modelling

To estimate the likelihood of pixels belonging to the foreground or background, colour distributions are learned from the user-provided scribbles.

In this work, the colour samples from the scribbles are used to build statistical models of foreground and background appearance. Each pixel is assigned a probability of belonging to either class based on its colour similarity to these models.

The unary data cost is computed as:

$$D_p(L_p) = -\log P(I_p|L_p)$$

where:

- I_p is the colour vector of pixel p
- L_p is the assigned label (foreground or background)
- $P(I_p|L_p)$ represents the likelihood of observing colour I_p given the label L_p

Pixels whose colours closely match the foreground distribution receive lower foreground costs, while pixels similar to the background distribution receive lower background costs.

5 Segmentation Algorithm

The Graph Cut segmentation procedure follows a sequence of steps that transform the input image and user annotations into a graph representation and compute the optimal segmentation using a min-cut/max-flow algorithm. The algorithm uses the user-provided scribbles to estimate foreground and background likelihoods for each pixel and then enforces spatial consistency through graph connections between neighbouring pixels.

The segmentation process is carried out as follows:

Algorithm 1 Graph Cut Segmentation

1: Input image and user scribbles
2: Estimate foreground and background colour models
3: Compute unary data costs for each pixel
4: Construct graph with terminal edges and neighbour edges
5: Apply max-flow/min-cut algorithm
6: Extract foreground mask
7: Apply post-processing (morphological filtering and smoothing)
8: Optionally update appearance models and repeat segmentation

After the graph is constructed, the max-flow/min-cut algorithm finds the optimal partition of the graph into two disjoint sets corresponding to foreground and background. The cut that minimises the energy function determines the final pixel labeling. The result is a binary mask that clearly separates the foreground object from the background.

5.1 Iterative Refinement

To improve segmentation accuracy, the graph cut optimisation can be performed iteratively. After an initial segmentation is obtained, the foreground and background models can be updated using the newly segmented regions.

The process involves the following steps:

- Estimate foreground and background appearance models from the current segmentation
- Recompute unary data costs for all pixels
- Reconstruct the graph with updated costs
- Reapply the min-cut/max-flow algorithm

This iterative procedure allows the segmentation to gradually improve as the appearance models become more accurate.



Figure 4: Raw Graph Cut segmentation results. Each row shows the original image (left), the binary segmentation mask (middle), and the overlay of the segmented foreground on the original image (right). White regions in the mask represent foreground, while black regions represent background. The overlay highlights the extracted foreground in green.

6 Results

The segmentation results demonstrate that the Graph Cut method effectively separates foreground objects from the background across different types of images. By combining

pixel-level colour information with spatial smoothness constraints, the algorithm is able to produce segmentation masks that closely follow the true object boundaries while reducing noise in homogeneous regions.

The incorporation of user-provided scribbles allows the algorithm to learn representative foreground and background characteristics, which significantly improves the quality of the segmentation. As a result, the method is capable of handling images with varying colour distributions, textures, and background complexities. Even in cases where the object boundaries are not clearly defined, the smoothness constraints help maintain spatial consistency in the segmentation.

Overall, the Graph Cut framework provides a balanced solution that integrates both local pixel information and neighbourhood relationships, leading to more accurate and visually coherent segmentation results.

The method achieves the following advantages:

- Smooth and continuous boundaries around the segmented object
- Reduced noise compared to simple pixel-wise segmentation methods
- Improved separation between foreground and background regions
- Robust performance across images with varying colours and textures



(a) Final Result 1 (b) Final Result 2 (c) Final Result 3

Figure 6: Final segmentation results obtained using the Graph Cut method for three different images.

7 Post-Processing

Although the Graph Cut algorithm produces high-quality segmentation masks, small artifacts may still remain near object boundaries or in regions with ambiguous colour information. To address these issues, several post-processing operations are applied.

Morphological filtering is used to remove isolated noise pixels and small background fragments. Hole-filling techniques ensure that segmented foreground regions remain

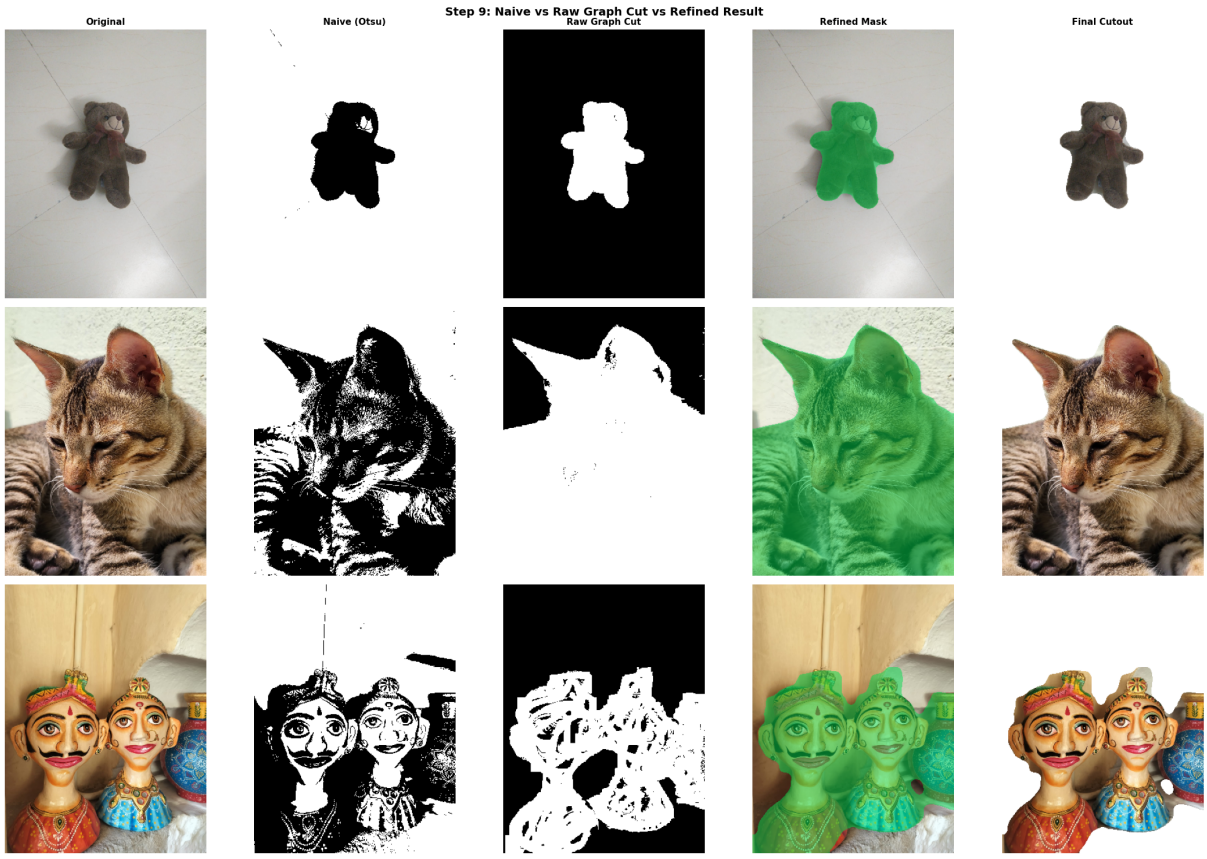


Figure 5: Comparison of segmentation stages. Each row shows a different example image, while the columns illustrate the processing pipeline: (1) Original image, (2) Naive Otsu thresholding, (3) Raw graph cut segmentation, (4) Refined mask overlay, and (5) Final object cutout. The refined graph cut approach produces cleaner and more accurate foreground extraction.

continuous and complete. Finally, boundary smoothing is applied to reduce jagged edges and improve the visual quality of the extracted object.

These refinement steps significantly enhance the final segmentation output.

8 Discussion

The Graph Cut based segmentation approach provides a strong balance between accuracy and computational efficiency. By combining colour-based data terms with spatial smoothness constraints, the algorithm produces globally consistent segmentation results.

However, the quality of segmentation strongly depends on the quality of user-provided scribbles. If the scribbles do not sufficiently represent the appearance of the foreground and background regions, the resulting segmentation may contain errors. Additionally, objects with very similar colours to the background can still present challenges.

Despite these limitations, Graph Cut segmentation remains a powerful and widely used technique for interactive image segmentation tasks.

9 Conclusion

In this report, we implemented an interactive image segmentation system based on the Graph Cut algorithm. The method formulates segmentation as an energy minimization problem that balances data fidelity and spatial smoothness.

Using user-provided scribbles, the algorithm constructs a graph representation of the image and computes the optimal segmentation using the min-cut/max-flow algorithm. Experimental results demonstrate that the method effectively extracts foreground objects while maintaining smooth and accurate boundaries.

Future improvements could include incorporating more advanced appearance models such as Gaussian Mixture Models and integrating automatic initialization techniques to further reduce user interaction.